Team Elha2

Elha2: A Food Saving App for Egypt

Software Design Document

Version 1.1

Name(s): Malak Dessouky
Yassin Shamaa
Ali Montasser
Farida Fayek
Ibrahim Meligy

Course: Software Engineering, Fall 2026

Date: 29/10/2026

American University in Cairo

Structure: SDD template summarised from IEEE Std 1016 (Recommended Practice for Software Design Descriptions).

Contents

Revision History **ii**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Overview . . . 1
1.4 Reference Material . . . 2
1.5 Definitions and Acronyms . . . 2

2 System Overview **4**
2.1 Functionality . . . 4
2.2 Context . . . 4
2.3 Design Summary . . . 5
2.4 Design Stakeholders and Concerns . . . 5

3 System Architecture **6**
3.1 Architectural Design . . . 6
3.2 Decomposition Description . . . 13
3.3 Design Rationale . . . 23

4 Data Design **25**
4.1 Data Description . . . 25
4.2 Data Dictionary . . . 28

5 Human Interface Design **31**
5.1 Overview of User Interface . . . 31
5.2 Screen Images . . . 32
5.3 Screen Objects and Actions . . . 32

6 Requirements Matrix **35**
6.1 Components to Requirements . . . 35
6.2 Functional Requirements to Components and Data . . . 36

7 Appendices **39**
Appendix A: Declaration of AI Use . . . 39

Revision History

Every sprint adds a row here. Text added or changed in a version is printed in blue with the version in the margin, and removed text is struck through.

Name	Date	Reason For Changes	Version
Elha2 team	10/10/2026	Milestone 1 baseline in the course SDD template: high-level architecture with UML component diagrams, architectural and design patterns, quality attributes, class and sequence diagrams, data dictionary, low-fidelity wireframes and requirements matrix	1.0
Elha2 team	29/10/2026	Architecture clarified: specific architectural style (modular monolith with one ML microservice); UML deployment diagram and failure table showing how the architecture supports availability and the other NFRs; table of specialised AI and ML models; context engineering in the AI gateway	1.1

1. Introduction

1.1 Purpose

This Software Design Document (SDD) explains how we will build Elha2, release 1.0. It shows the main parts of the system, what each part does, how the parts work together, what data we store, and what the screens look like. It turns the requirements in the Elha2 Software Requirements Specification (SRS) [1] into a design.

We update it at the end of every sprint together with the SRS and record each change in the revision history.

1.2 Scope

Elha2 is an Arabic-first mobile app and web portal, helped by AI, that reduces food waste in Egypt in three ways: *Prevent* (a smart pantry that fills itself from receipts, photos and voice notes and warns before food spoils), *Sell* (businesses sell surplus food at a discount for pickup) and *Donate* (unsold food goes to NGOs, and households can donate sealed items). Every confirmed saving is measured in kilograms, Egyptian pounds, CO_2e and meals.

Our goals are to cut household food waste, make tracking food effortless, turn business surplus into affordable food, get food to people in need, and measure the impact honestly, while keeping food, users and data safe (SRS Section 1.4). Users save money, businesses recover revenue, NGOs get a steady supply of safe food, and partners get reliable data for SDG 12.3 reporting.

The architecture and data design (Chapters 3 and 4) cover all 14 features in the SRS, so the design does not have to change when later features are added. The screens (Chapter 5) cover the 39 stories we build during the course (SRS Section 6.6) and grow sprint by sprint.

1.3 Overview

Chapter 2 describes what the system does and who it serves. Chapter 3 presents the architecture: the main parts and their jobs, the specific architectural style and patterns we use, how the architecture supports each quality requirement (with a deployment view), the AI models and how their input is built, how the parts talk to each other, and why we chose this design. Chapter 4 shows how data is stored, with UML class diagrams and a data dictionary. Chapter 5 shows the user interface as simple wireframes. Chapter 6 links every part of the design to the SRS requirements it satisfies. The appendix declares how we used AI in writing this document.

1.4 Reference Material

[1] Elha2 team, *Elha2 Software Requirements Specification, Version 1.0*. American University in Cairo, 2026.

[2] IEEE, *IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions*. New York: IEEE, 2009.

[3] Object Management Group, *Unified Modeling Language (UML), Version 2.5.1*, 2017.

[4] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[5] L. Bass, P. Clements and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.

[6] Google, *Material Design 3 Guidelines*, m3.material.io.

1.5 Definitions and Acronyms

Requirement terms are defined in SRS Appendix A. The terms below are used in this design.

Term	Meaning
Adapter	A small class that wraps an outside service (for example Paymob) so the rest of our code does not depend on that service directly.
AI gateway	The one backend part that every request to an AI model goes through.
API	Application programming interface: the set of web addresses (endpoints) the apps call to read and save data.
Backend	The server program that holds the business rules and talks to the database.
Component	A self-contained part of the system with a clear job. In UML diagrams it offers services to other parts and uses services from them.
Impact ledger	The part that records each confirmed saving (kg, EGP, CO_2e, meals) once and never edits it afterwards.
Interface	A named list of operations one part offers to the others, for example `IStock.reserve()`.
Modular monolith	One backend program split into separate modules. Each module has its own job and data and talks to other modules only through interfaces.
NestJS	A TypeScript framework for building backends on Node.js.
ORM	Object-relational mapper: a library that turns code classes into database tables (we plan to use Prisma).
pg-boss	A library that stores scheduled background jobs in PostgreSQL.
PostGIS	An add-on for PostgreSQL that handles locations and distances.
Riverpod	A Flutter library that keeps the screens up to date when data changes.
Supabase	A hosted platform built on PostgreSQL. We use it for the database, phone login (Auth), file storage (Storage) and live updates (Realtime).

Term	Meaning
Supabase Realtime	A Supabase service that keeps a connection open so the backend can push live updates (for example “2 bags left”) to the apps.

2. System Overview

2.1 Functionality

Elha2 serves six kinds of user through two apps. Households use the **mobile app** to keep a smart pantry (filled from receipts, barcodes, photos and voice notes, and checked once a week), get expiry reminders, cook suggested Egyptian recipes, buy discounted surplus food, donate sealed food to NGOs and see how much they have saved. Businesses, NGOs, administrators and government analysts use the **web portal** to list surplus and confirm pickups, receive donation offers, run the platform and export anonymised data. These functions are grouped into 14 features (E1 to E14) and written as 100 user stories in SRS Chapter 4.

2.2 Context

Elha2 is a new, standalone product. Figure 2.1 shows the people and organisations it serves and the outside systems it uses for login, notifications, maps, payments, AI models, product data, weather and grocery emails. Our code reaches each outside system through an adapter or the AI gateway, so any of them can be replaced without changing the rest of the code.

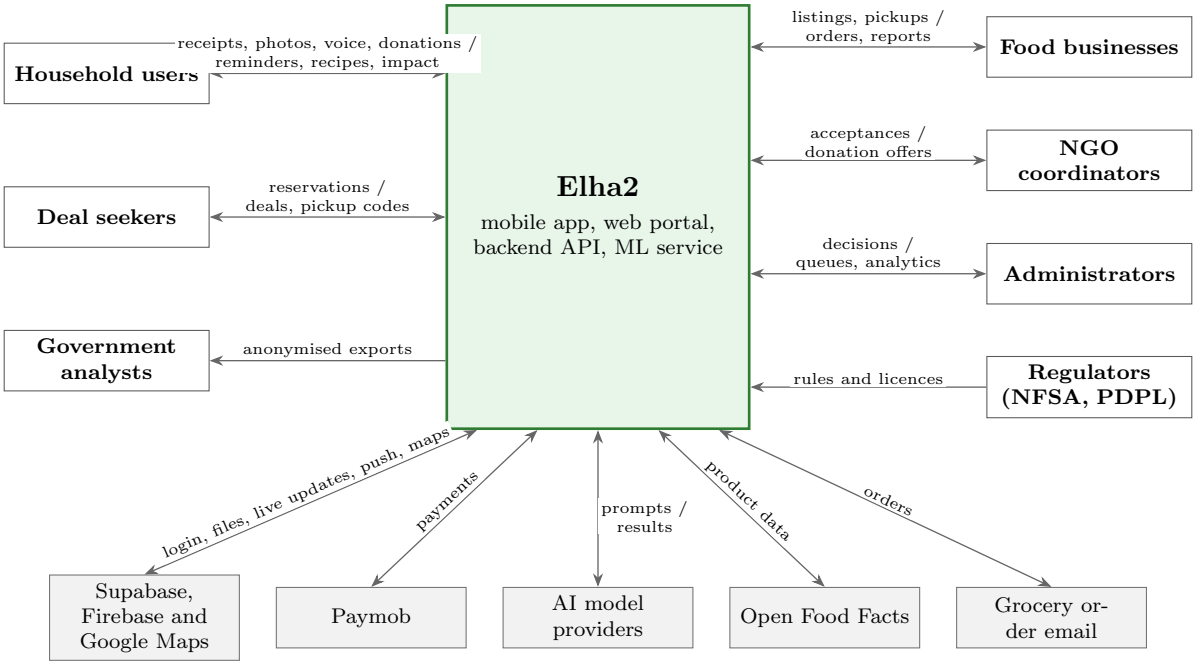


Figure 2.1: Context diagram: the Elha2 system boundary, the people and organisations it serves, and the external systems it depends on. Labels show the data sent in / out.

2.3 Design Summary

Elha2 is a client–server system with three tiers. Two clients, a Flutter mobile app and a React web portal, talk to one NestJS backend over the internet. More specifically, the backend is a *modular monolith*: a single program split into eight feature modules and five shared services, which talk to each other only through named interfaces. Machine learning runs in one separate *microservice* written in Python. One PostgreSQL database, hosted on Supabase, stores all the data, including locations and scheduled jobs. Supabase also provides phone login, file storage and live updates; Firebase is used only for push notifications and to host the web portal. Text and barcode reading happen on the phone. Every request to a hosted AI model goes through one AI gateway, which checks the answer and switches to a non-AI option if the model fails. Chapter 3 explains this design.

2.4 Design Stakeholders and Concerns

Each stakeholder's main concern is handled by at least one part of the design.

Stakeholder	Design concern	Handled by
Household users	Logging food with almost no typing; works offline; personal data stays private	AI subsystem (Section 3.2.2); text reading and data masking on the phone; offline pantry (Section 3.2.3)
Business staff	Fast listing; smooth pickup; never selling the same bag twice	Marketplace and Orders modules (Section 3.2.1); safe stock update (Section 4.1)
NGO coordinators	Predictable donation offers that match what they can accept	Donations module and Scheduler (Section 3.2.5)
Administrators	Tools to verify and moderate; rules that can change without new code	Administration module; settings stored in the database
Government partners	Trustworthy, anonymised impact data	Impact ledger; data design (Chapter 4)
Outside service providers	Staying within usage limits and terms of use	AI gateway cost limit and cache; adapters

3. System Architecture

3.1 Architectural Design

3.1.1 Subsystems and Responsibilities

We split the system by *who uses it* and *what kind of work it does*, following the features and stories in the product backlog (SRS Chapter 4):

- People on phones (households and deal seekers) use the **mobile app**; organisations on computers (businesses, NGOs, administrators, analysts) use the **web portal**.
- All business rules live in one **backend**, so both apps behave the same way and each rule is written only once.
- Machine learning needs Python libraries and long training runs, so it has its own **ML service**.
- Outside services (payments, maps, AI models) are always reached through our own interfaces, so we can swap them.

Figure 3.1 shows these subsystems, the database and the connections between them as a UML component diagram. The table after it gives each component's job.

Component	Responsibility	Technology	SRS trace
Mobile app	All household and deal seeker features, including household donations; reads receipt text and barcodes on the phone and hides personal data before upload; works offline for the pantry	Flutter, ML Kit, SQLite (Drift), Riverpod	E1–E5, E7, E8–E11; NFR-16, NFR-21, NFR-27
Web portal	Business listings and pickups, NGO offers, administration, government exports	React, TypeScript	E6, E8, E9, E13
Backend API	Business rules, security and data access for both apps; receives updates from the payment and email providers	NestJS (Node.js, TypeScript)	All features; NFR-02, NFR-09, NFR-10
ML service	Estimates shelf life and recommends deals; retrains every night	Python, FastAPI, scikit-learn	US-3.01, 5.06
Supabase database	Stores all data, locations and scheduled jobs; answers "what is near me" queries	Supabase PostgreSQL with PostGIS, pg-boss	SRS Section 6.1; NFR-03, NFR-20
Supabase services	Phone-number login, photo and document storage, live updates to the apps	Supabase Auth, Storage, Realtime	US-1.01, 5.03, 6.02

Component	Responsibility	Technology	SRS trace
Firebase and Google Maps	Push notifications; maps and addresses	Firebase Cloud Messaging, Google Maps Platform	US-5.01, 11.01
AI model providers	Language, vision and speech models, reached only through the AI gateway	Hosted APIs (e.g. Claude, GPT or Gemini; Whisper)	Stories marked [AI]; NFR-07, NFR-22, NFR-29, NFR-30
Payment provider	Card and wallet payments, refunds, payouts	Paymob	E7; NFR-11
Open Food Facts, grocery email	Product details for barcodes; forwarded grocery orders	REST API, email webhook	US-2.03, 2.04

3.1.2 How the Subsystems Work Together

- **Apps to backend.** The mobile app and web portal call the backend's REST API to read and save data. Users log in with Supabase Auth, and each request carries their Supabase login token, which the backend checks first. For live updates such as bags left and order status, the apps listen on Supabase Realtime channels, and the backend posts to those channels. The apps never read or write the database directly.
- **Inside the backend.** Each request enters through the API layer and is handled by one feature module (Section 3.2.1). Feature modules use the shared services (AI gateway, impact ledger, notifications, scheduler, adapters) and, when needed, each other, always through named interfaces.
- **Backend to ML service.** The backend asks the ML service for shelf-life estimates and recommendations over an internal web call. The ML service can only read its training data; it never changes business data.
- **Backend to outside services.** Payments, maps, product data, grocery emails and file storage (Supabase Storage) go through adapters. AI models go through the AI gateway. Push notifications go through Firebase Cloud Messaging.

3.1.3 Architectural Style: a Modular Monolith with One ML Microservice

Client–server only says that the apps send requests to a server; it does not say how the server side is built. The specific style of Elha2 is:

- **A modular monolith for the backend.** All business rules run in one NestJS program, split into eight feature modules and five shared services (Section 3.2.1). Each module owns its own tables and talks to the others only through named interfaces. The backend is deployed as one unit and scaled by running more identical copies of it (Figure 3.2).

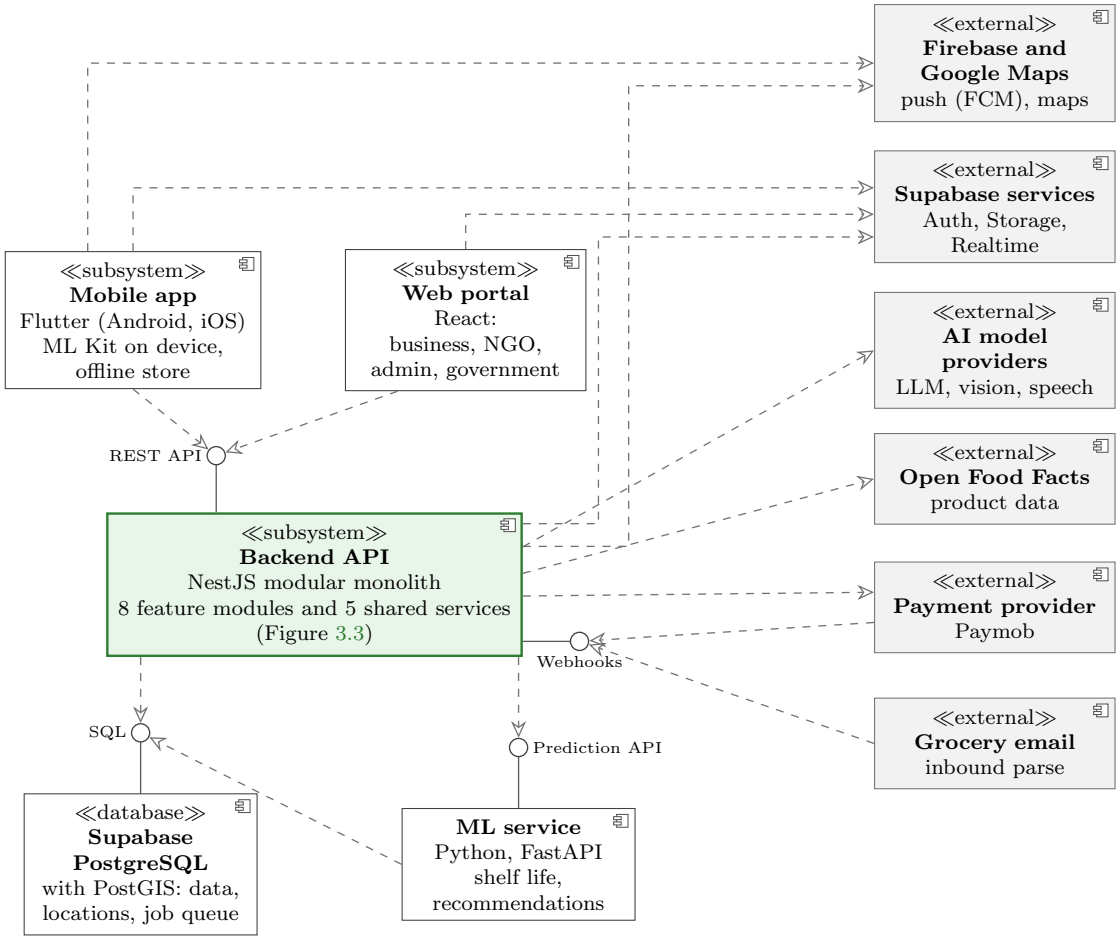


Figure 3.1: UML component diagram 1: the whole system. A ball (circle) is an interface a component offers; a dashed arrow means "uses"; grey components are outside services.

- **One separate microservice for machine learning.** The ML service is its own Python program with its own deployment, scaling and release cycle, and the backend reaches it only through its Prediction API. If it is down, the backend falls back to rule-based estimates, so the rest of the system keeps working.
- **A gateway in front of all hosted AI models** (Section 3.2.2), so no feature module calls an AI provider directly.
- **Layers inside each part.** The backend and the mobile app are each built in layers (Sections 3.2.1 and 3.2.3).

We chose this over full microservices because a team of five cannot run and keep consistent a dozen services in one semester (Section 3.3). The module interfaces already mark where the system would split: if one module's load grows much faster than the rest (most likely Marketplace search or Notifications), it can be moved into its own microservice without changing the other modules.

3.1.4 Architectural and Design Patterns

A pattern is a well-known, tested solution to a common design problem. The table lists the patterns we use, where we use them and why.

Pattern	Where we use it	Why (quality and SRS trace)
Architectural patterns (the overall shape of the system)		
Client–server (outer shape)	Two clients, the mobile app and the web portal, send requests to one server side. With the database this gives three tiers: the apps show data, the backend and ML service apply the rules, the database stores the data	Rules are checked once on the server, and each client can change without touching the others (NFR-10, NFR-27)
Layered architecture	Used in two places. **Backend:** API layer → feature modules → shared services → database (Section 3.2.1). **Mobile app:** screens → state → data → phone features (Section 3.2.3)	Each layer only uses the layer below it, so parts are easy to test and replace (NFR-24)
Modular monolith	The backend: eight feature modules inside one NestJS program, each owning its own data	Easy for a team of five to build and run, while each module could later become its own service (NFR-24)

Pattern	Where we use it	Why (quality and SRS trace)
Microservice	The ML service: a separate Python program, deployed and scaled on its own, reached only through its Prediction API	ML work can use Python libraries and heavy training jobs without slowing the backend; a failure there does not stop the backend (NFR-05, NFR-22)
Design patterns (inside the code) [4]		
Facade	The AI gateway is the single front door to all AI models; the API layer is the single front door to all modules	Prompts, answer checks, caching, cost limits and fallbacks are handled in one place (US-14.01, 14.05)
Adapter	Paymob, Google Maps, Open Food Facts, grocery email and the ML service are each wrapped in an Elha2 interface	We can change a provider without changing feature code (R-09, NFR-11)
Strategy	The AI provider and its fallback are chosen per task in settings; the payment method (cash or Paymob) sits behind `IPayment`	A new provider or payment method is added as a new strategy, without rewriting existing code (NFR-22, E7)
Repository	Each backend module and the mobile app's data layer read and write data through repository classes	Screens and business rules never write SQL or call the network directly (NFR-21, NFR-24)
Template method	Every AI task follows the same steps (build prompt, call model, check answer, log, fall back); only the prompt and expected answer format change per task	All AI features get the same safety checks and logging (NFR-22, NFR-30)

3.1.5 Quality Attributes: How the Architecture Supports the NFRs

Each non-functional requirement in SRS Chapter 5 is supported by a specific part of the architecture. Figure 3.2 is a UML deployment diagram: it shows where each part runs, and its green tags mark the NFR that each part helps to meet.

Availability (NFR-23: 99.5% of each month). The target allows about 3.6 hours of downtime a month. The architecture meets it in three ways. First, there is *no single copy* of the backend: at least two identical API instances run behind a load balancer, and because they keep no user state, either one can answer any request. Second, *failures are detected and repaired automatically*: the load balancer checks every instance every 10 s, and the platform restarts a failed one. Third, *when a part is down, the rest keeps working* (graceful degradation), as the table shows. Monitoring alerts the team whenever errors pass 2% (NFR-26).

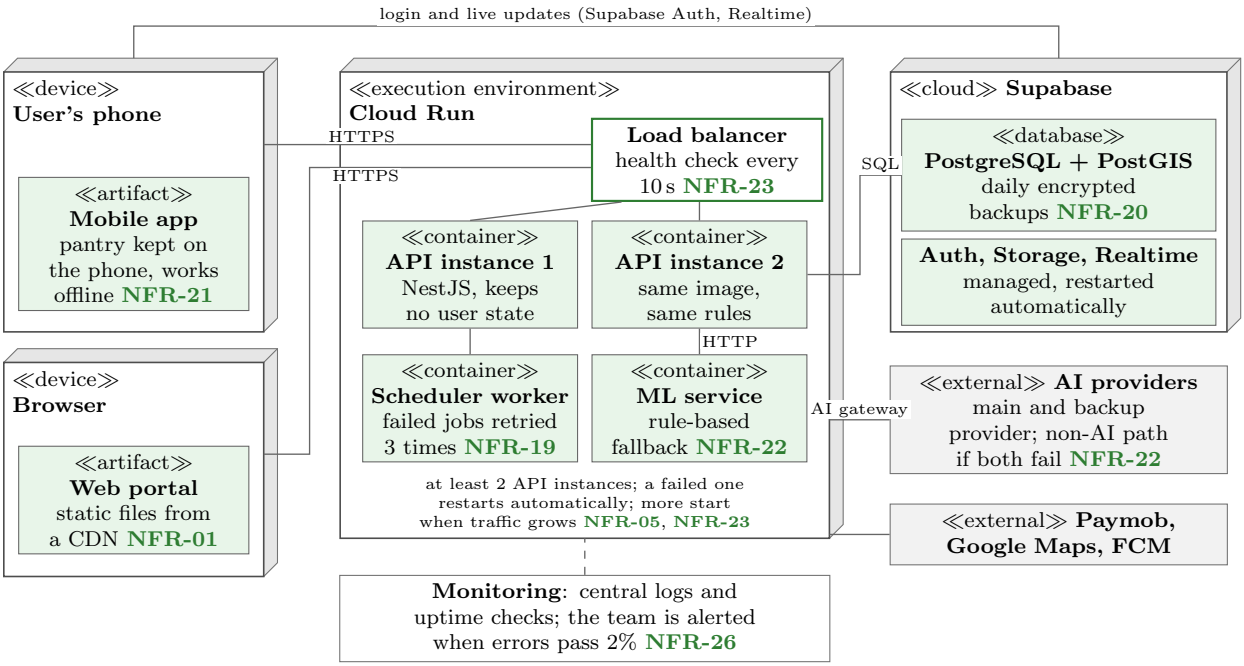


Figure 3.2: UML deployment diagram: where each part runs and which NFR it supports (green tags). Boxes drawn in 3-D are nodes (devices or execution environments); the shaded boxes inside them are what runs there.

If this fails	What the architecture does	What users notice
One API instance	The load balancer stops sending it requests within about 10 s; the other instance answers; the platform starts a replacement	Nothing
Deploying a new version	Rolling deployment: new instances must pass the health check before old ones stop	Nothing; no downtime
The ML service	The Expiry module uses the rule-based shelf-life table; deals are ordered by distance	Slightly less personal estimates and deals
An AI provider (or slower than 15 s)	The AI gateway retries once, switches to the backup provider, then to the non-AI path (NFR-22)	Items are confirmed or typed by hand
The database	Supabase restarts it automatically; daily encrypted backups restore it within 4 h, losing at most 24 h (NFR-20)	The pantry still works offline on the phone and syncs later
A scheduled job	The job is retried 3 times, then the team is alerted (NFR-19)	A reminder or offer may arrive a few minutes late
The phone's connection	The pantry is kept on the phone; changes are queued and sent later (NFR-21)	The pantry works; deals need a connection

All quality attributes. The table links every group of NFRs to the architectural element that supports it.

Quality attribute	Architectural element	How it supports the NFR	NFRs
Performance	On-device models; PostGIS index; AI gateway cache; CDN	Text and barcodes are read on the phone without an upload; "near me" searches use a location index; repeated AI answers are cached; the portal is served from a CDN	NFR-01 to NFR-04
Scalability	API containers; Scheduler worker; ML microservice	The backend keeps no user state, so more copies start automatically when traffic grows; slow work runs in background jobs; the ML service scales on its own	NFR-05
Integrity	Marketplace module; Impact ledger; database	Stock is reduced in one database step, so a bag cannot be sold twice; each saving is recorded once	NFR-06
Availability	Load balancer with 2+ API instances; managed Supabase; fallbacks	See the failure table above	NFR-23
Reliability	Scheduler worker; AI gateway; mobile offline store; Supabase backups	Jobs are retried; every AI task has a non-AI path; the pantry works offline; daily backups	NFR-19 to NFR-22
Security and privacy	API layer guard; on-device masking; Supabase encryption	Login token and role checked on every request; personal data hidden before upload; data encrypted in transit and at rest; no card data stored	NFR-09 to NFR-15
Safety	AI gateway checks; Marketplace screening; Administration	AI answers pass a food-safety filter; allergens are checked and listings screened; only verified businesses can list	NFR-07, NFR-08
Usability	Mobile app screens; review screen	Arabic-first right-to-left layout; one-tap review of AI results with a confidence	NFR-16 to NFR-18, NFR-30
Maintainability	Module interfaces; adapters; monitoring	Clear boundaries, swappable providers, automated tests on every merge, central logs and alerts	NFR-24 to NFR-26
Portability	Flutter app; responsive React portal	One code base for Android and iOS; the portal runs in current browsers	NFR-27, NFR-28

3.1.6 Technical Feasibility

The design is realistic for our team within one semester. Every part uses mature, well-documented tools (Flutter, React, NestJS, Supabase, Firebase) with free tiers or student credits. Supabase gives us the database, login, file storage and live updates from one service, so we have fewer services to set up. We do not train large AI models: we use existing hosted models and on-device libraries (SRS Section 2.2.1), and train only small models on tabular data. The clear interfaces between parts let team members work on different parts at the same time.

3.2 Decomposition Description

We describe the system in an **object-oriented** way, as parts with clear jobs and interfaces. Section 3.2.1 breaks down the backend, Section 3.2.2 the AI and ML parts and Section 3.2.3 the two apps. Section 3.2.4 lists the interfaces between them, and Section 3.2.5 shows how they work together step by step. The class diagrams are in Chapter 4.

3.2.1 Backend API Subsystem

The backend follows the *layered architecture* pattern (Section 3.1.4): it has three layers, and each layer only uses the layer below it (Figure 3.3). The *API layer* receives every request, checks the login token and checks the input. The eight *feature modules* hold the business rules, one for each area of the backlog. The five *shared services* are used by all feature modules. Modules call each other only through named interfaces, so each one can be tested on its own and different team members can work on different modules.

Module	Responsibility	Features	Owns data
Feature modules			
Accounts	Sign-up, profiles, households and members, organisations, branches, staff roles	E1	User, Household, Business, Branch, NGO
Pantry	Pantry items, receipt drafts, weekly check, status changes, waste reasons, daily expiry reminders	E2, E3	PantryItem, ReceiptDraft
Expiry	Shelf-life estimates (using the ML service) and label types	E3	FoodCatalogueItem
Recipes	Recipe collection, ranking by food about to expire, AI-written backup recipes, cooking assistant	E4	Recipe
Marketplace	Listings, stock, search by distance, automatic listing checks	E5, E6, E12	Listing
Orders	Reservations, pickup codes, payments, refunds, payouts, no-show rule	E7	Order, Payment
Donations	NGO preferences, matching, offers, pickups, household donations, meals served	E8	DonationOffer
Administration	Verification, moderation, disputes, settings, platform statistics	E12, E13	Report, ModerationCase, Setting
Shared services			
AI gateway	Single entry point to all AI models (Section 3.2.2)	E14	PromptTemplate, AIRequestLog
Impact ledger	Records each confirmed saving once and never changes it; turns kg into EGP, CO_2e and meals; feeds dashboards and reports	E9	ImpactRecord, ImpactFactor
Notifications	Push and in-app notifications, live updates through Supabase Realtime, user preferences, quiet hours	E11, US-5.03	Notification, DeviceToken

Module	Responsibility	Features	Owns data
Scheduler	Timed background jobs: daily reminders, end-of-pickup donations, no-shows, nightly model training	E3, E8	pg-boss job tables
Adapters	Wrappers for Paymob, Google Maps, Open Food Facts, grocery email, Supabase Storage and the ML service	E2, E6, E7	–

Module dependencies. The table lists the interfaces each module uses. Dependencies only go from feature modules to shared services, or between feature modules as drawn in Figure 3.3, so no two modules depend on each other in a loop.

Module	Interfaces it uses
Accounts	INotify; IExternal (Supabase Auth)
Pantry	IEstimateExpiry, IAI, IRecordImpact, INotify; IExternal (IProduct, IInbound)
Expiry	IExternal (IPredict)
Recipes	IPantryQuery, IAI
Marketplace	IAI, INotify; IExternal (IGeo, IPredict)
Orders	IStock, IRecordImpact, INotify; IExternal (IPayment)
Donations	IStock, IRecordImpact, INotify; IExternal (IGeo)
Administration	IOrgVerify, INotify; IExternal (IPayment, for refunds)
Scheduler	Pantry (reminders), Marketplace (end of pickup window), Donations (offers), Orders (no-shows)

3.2.2 AI and ML Subsystem

Elha2 does not rely on one large language model. Most of its AI work is done by **small, specialised models**, each chosen for one job, several of which run on the phone or in our own ML service. A general LLM is used only where the input is too varied for a fixed model (messy receipt lines, voice transcripts, cooking questions), and always behind the checks of the AI gateway. Figure 3.4 shows where each model runs, and the table lists every model and its job.

Model	Kind	Runs on	Job	Stories
Specialised models				
ML Kit text recognition	Pre-trained vision model	Phone	Reads receipt lines and printed dates, also offline	US-2.01, 3.02
ML Kit barcode scanning	Pre-trained vision model	Phone	Reads barcodes; details come from Open Food Facts	US-2.03
Personal-data masker	Rules for names, phone and card numbers	Phone	Hides personal data before anything is uploaded	US-14.03
Speech-to-text (e.g. Whisper)	Pre-trained speech model	Provider, via gateway	Turns Egyptian Arabic voice notes into text	US-2.06

Model	Kind	Runs on	Job	Stories
Item matcher	Multilingual text-embedding model plus fuzzy matching	ML service	Maps a line such as "TOM 1KG" or an Arabic name to a catalogue item	US-2.01, 2.06
Predictive models we train on our own data				
Shelf-life model	Gradient-boosted trees (scikit-learn), starting from food-storage rules	ML service	Predicts days until an item spoils from its category, storage place, season and past corrections	US-3.01
Weekly-check ranker	Uses the shelf-life model's uncertainty and the household's usage history	ML service	Picks the 10 items most likely to be wrong for the weekly check	US-2.09
Deal recommender	Matrix factorisation on past orders, re-ranked by distance and time of day	ML service	Orders each user's deal feed	US-5.06
Algorithms (no learning)				
Recipe ranker	Scoring of curated recipes	Recipes module	Ranks recipes by how many at-risk items they use	US-4.01
Route planner	Nearest-neighbour route with pickup windows	Donations module	Suggests the order of NGO pickups	US-8.03
General hosted models, always behind the AI gateway				
Vision-language model	General multimodal model	Provider, via gateway	Finds items in a fridge photo; fills a listing from a photo	US-2.05, 6.02
Large language model	General LLM (choice is SRS TBD-01)	Provider, via gateway	Turns messy lines and transcripts into structured items; cooking answers; monthly tips; backup recipes	US-2.01, 2.06, 4.02, 9.05

Every AI request follows the same rules: wait at most 15 s, retry once, check that the answer has the expected format, switch to the backup provider, and use a non-AI fallback if it still fails (NFR-22). The gateway logs the model, response time, cost and confidence, which feed the accuracy report (US-14.02) and the cost limit (US-14.05). The provider is set in configuration, so it can change without touching the feature modules.

Context engineering. The quality of a general model's answer depends mostly on what it is given. The *Context builder* in the AI gateway (Figure 3.4) therefore assembles the input for every LLM and vision call in the same six steps (Figure 3.5): it uses a versioned template, adds only the context that task needs, adds a few worked examples, hides personal data and trims the input to a size limit, asks for a structured (JSON) answer, and checks that answer before anyone sees it. Corrections that users make on the review screen are stored, without personal data, as new

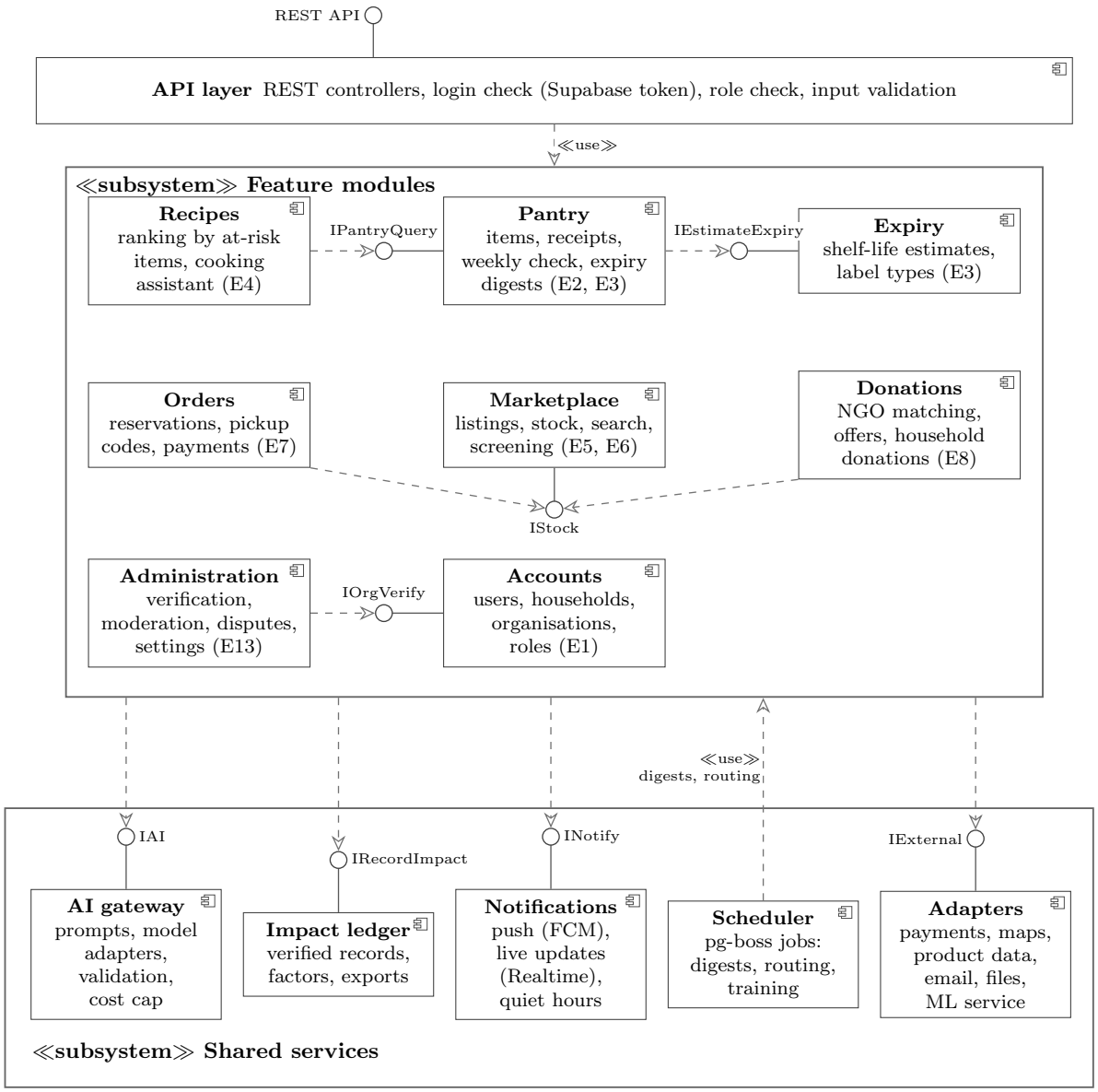


Figure 3.3: UML component diagram 2: inside the backend.

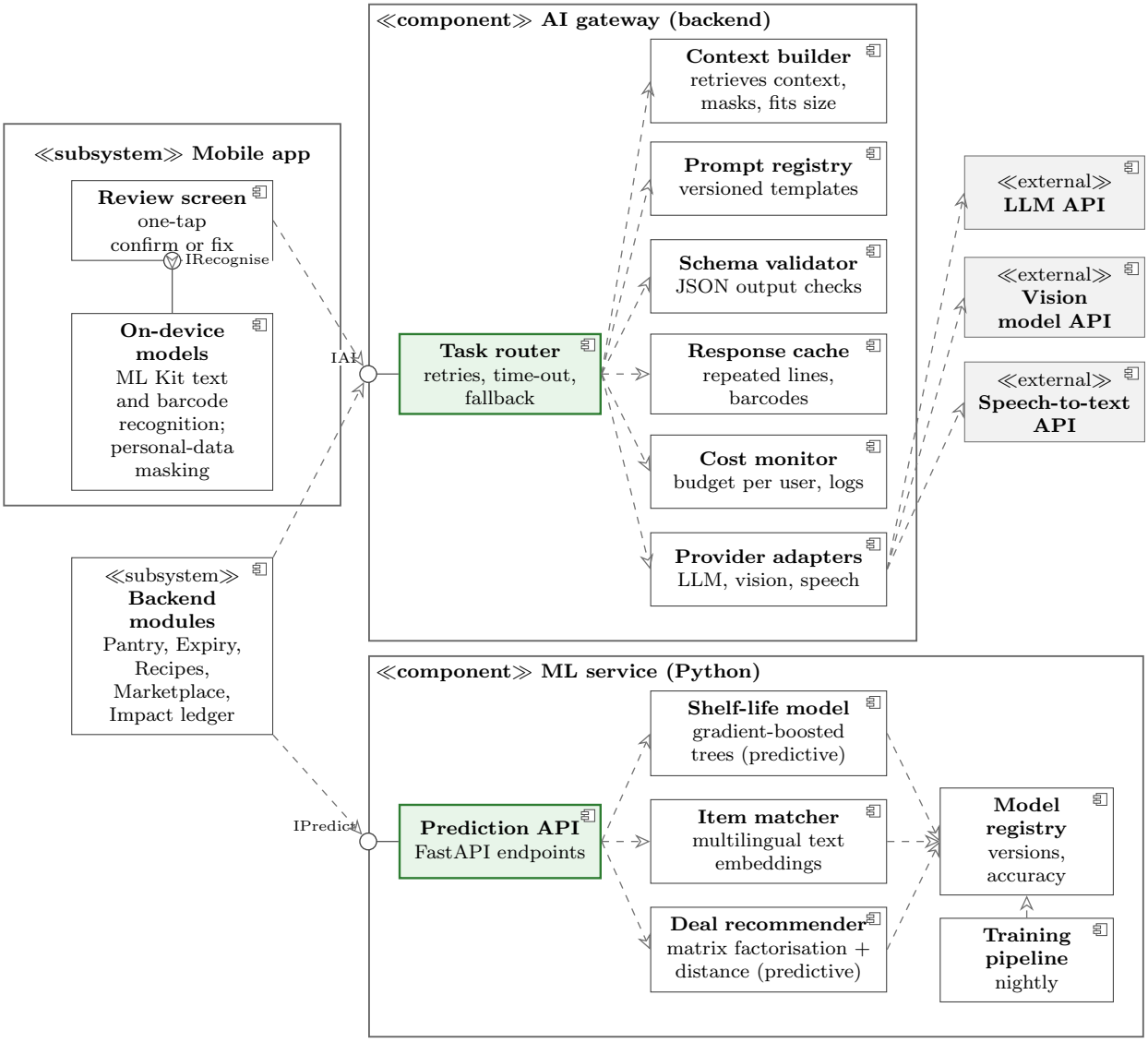


Figure 3.4: UML component diagram 3: the AI and ML parts on the phone, in the backend (AI gateway) and in the ML microservice.

examples, so the prompts improve over time. A changed template is released only after it is tested on the labelled test set (US-14.02).

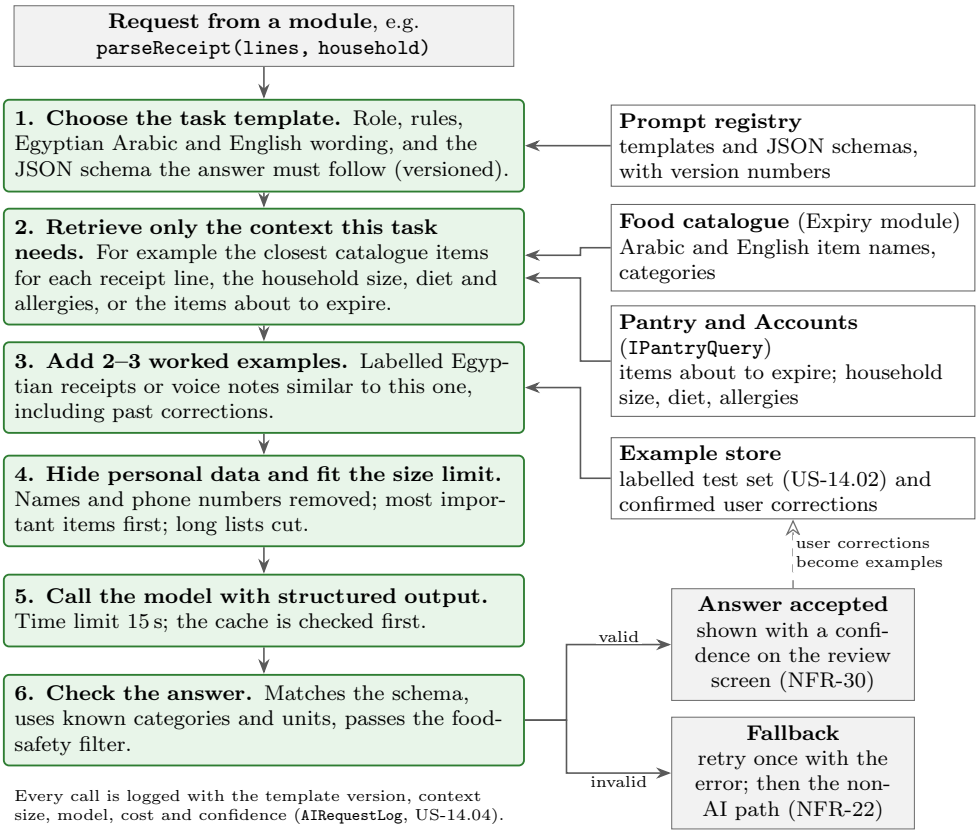


Figure 3.5: Context engineering in the AI gateway: the six steps that build and check the input and output of every general-model call.

Task	Context the model receives	Answer format	Never sent
Read a receipt	Masked receipt lines; the 5 closest catalogue items for each line; 2 similar labelled receipts; the shop name if known	List of items: name, catalogue ID, quantity, unit, confidence	The photo; names, phone or card numbers
Read a voice note	The transcript; the household's usual items; catalogue matches	Same list of items	The audio after transcription
Recognise a photo	The photo (deleted within 24 h); the list of allowed food categories	Items with counts and confidence	Location; who the user is
Cooking assistant	The question; items about to expire; household size, diet and allergies; food-safety rules	Short answer with a reheating note	Other users' data
Monthly insight	The user's own waste totals by food category	At most 3 tips, each citing one of the user's numbers	Item-by-item history

Task	Context the model receives	Answer format	Never sent
Backup recipe	Items about to expire; diet and allergies; an instruction to use Egyptian dishes	Recipe in a fixed format, then the safety filter	–

3.2.3 Mobile App and Web Portal

The mobile app also follows the *layered architecture* pattern: it has four layers, and each layer only uses the layer below it, so screens never call the network directly. This keeps the offline pantry and syncing in one place.

Layer	What it contains
Screens	The five tabs (Home, Pantry, Deals, Donate, Impact) and Settings, in Arabic and English with right-to-left layout
State	Riverpod providers that hold what each screen shows and react to user actions
Data	Repositories; API client for REST; Supabase client for login and live updates; offline database on the phone (SQLite through Drift); a queue of changes made offline, sent when the phone is back online
Phone features	ML Kit text and barcode reading and personal-data hiding; camera and microphone; push notifications (FCM); Google Maps; secure storage of the login token

The web portal follows the same idea: React pages, one shared API client, and separate pages for businesses, NGOs, administrators and government analysts.

3.2.4 Interface Specifications

External API (apps to backend). Both apps use one REST API (`/api/v1`) that sends and receives JSON. Every request carries the Supabase login token, which the API layer checks first; the user's role decides what they may do (NFR-10). Errors always have the same shape (code, message, field errors) and are shown in the user's language. Reservation and payment requests carry a unique request key, so if the app sends the same request twice (for example after a network drop), only one order is created. Live updates (bags left, order status) are sent through Supabase Realtime.

Resource	Main operations	Used by	Stories
`/me`, `/households`	Profile, language, location; household members and invitations	Mobile	E1
`/organisations`	Register a business or NGO, branches, staff, documents	Portal	US-1.06, 1.07, 6.03
`/pantry/items`	List, add, change status, delete; search	Mobile	US-2.07–2.11

Resource	Main operations	Used by	Stories
`/pantry/receipts`	Upload hidden-data receipt text, get draft items, confirm	Mobile	US-2.01, 2.02
`/pantry/check`	Get this week's check cards, send swipes	Mobile	US-2.09
`/recipes`	Suggestions for food about to expire, filters, mark as cooked	Mobile	E4
`/listings`	Search nearby (distance, filters), create, edit, cancel	Both	E5, E6
`/orders`	Reserve, cancel, history, confirm pickup by code	Both	E7
`/donations`	NGO offers, accept or decline, book pickup, record meals; household donations to drop-off points	Both	E8
`/impact`	Personal, business and NGO dashboards; how we calculate	Both	E9
`/exports`	Company sustainability report (PDF), anonymised government data (CSV)	Portal	US-9.08, 9.10
`/admin`	Verification, moderation, disputes, settings, statistics	Portal	E12, E13
`/webhooks`	Payment updates (Paymob), forwarded grocery emails	Providers	E7, US-2.04

Internal interfaces (between backend parts).

Interface	Offered by	Main operations	Used by
IPantryQuery	Pantry	`atRiskItems(household, days)`, `deduct(items)`	Recipes
IEstimateExpiry	Expiry	`estimate(item, storage, purchased)` returns a date, where it came from, and a confidence	Pantry
IStock	Marketplace	`reserve(listing, qty)` (in one database step), `release(order)`, `unsoldAtWindowEnd(listing)`	Orders, Donations
IOrgVerify	Accounts	`setVerification(org, status, reason)`	Administration
IAI	AI gateway	`parseReceipt`, `parseVoice`, `recognisePhoto`, `suggestRecipe`, `summariseWaste`; each returns a result, a confidence and whether the fallback was used	Pantry, Recipes, Marketplace, Impact ledger
IRecordImpact	Impact ledger	`record(sourceAction, items, parties)`; recording the same action twice has no extra effect	Pantry, Orders, Donations
INotify	Notifications	`send(user, type, payload)`, which respects user preferences and quiet hours	All modules
IExternal	Adapters	IPayment (charge, refund, check webhook), IGeo (address to location, distance), IProduct (barcode lookup), IFiles (upload, link, delete), IPredict (shelf life, recommend), IInbound (read email)	See Section 3.2.1
IPredict (HTTP)	ML service	`POST /shelf-life`, `POST /recommend`	Adapters

The outside services (Supabase, Firebase, Paymob, Google Maps, AI providers, Open Food

Facts, grocery email) are listed in SRS Section 3.3. Each is wrapped in an adapter, so the rest of the code never depends on a provider's library directly.

3.2.5 Sequence Diagrams

Scan a receipt (UC-05). Figure 3.6 shows how the components work together when a household user scans a receipt. The phone reads the text and hides personal data, so only safe text reaches the backend. The AI gateway asks the LLM to list the items, checks the answer and records the cost. The Pantry module then adds expiry estimates. Nothing is saved until the user confirms.

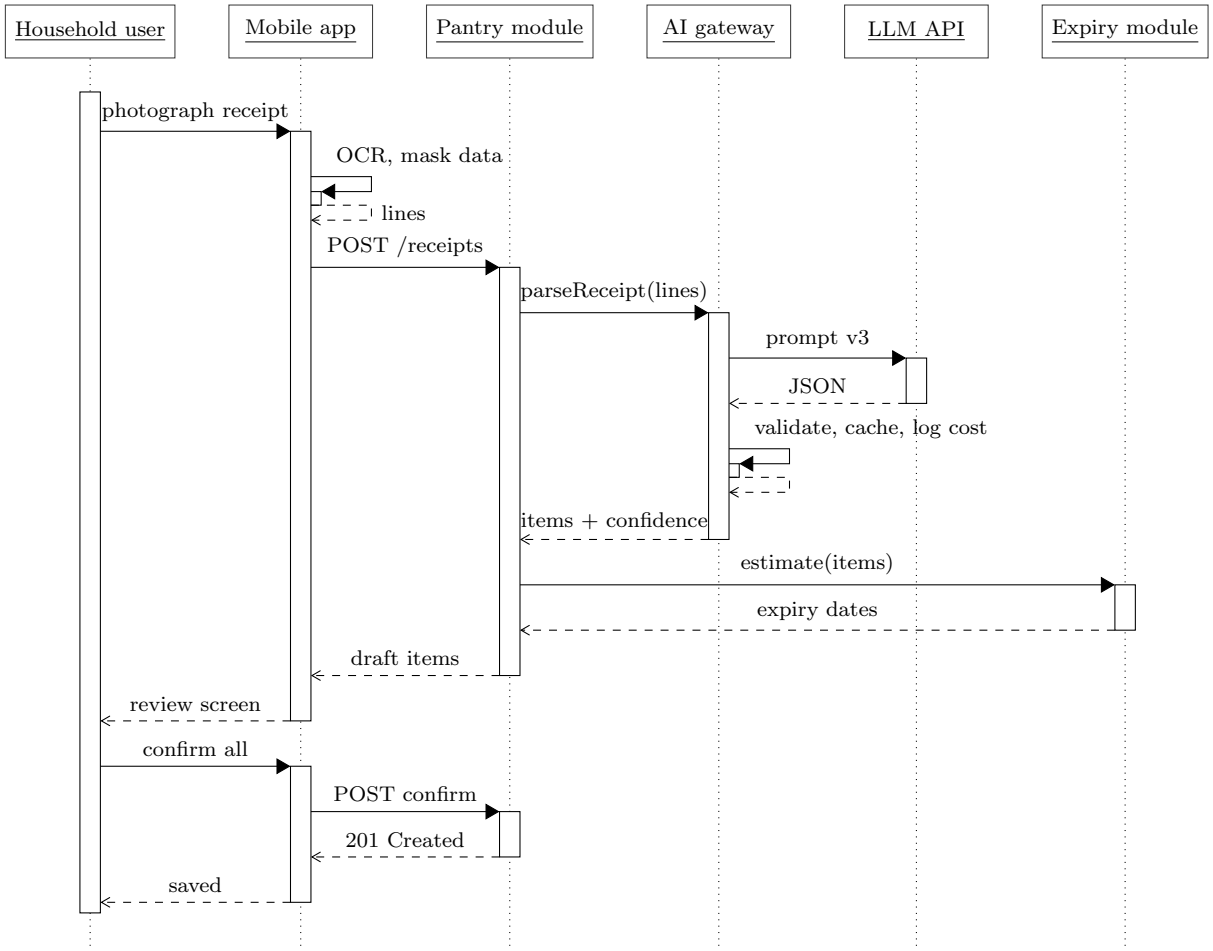


Figure 3.6: UML sequence diagram: scanning a receipt (SRS UC-05).

Reserve and collect a bag (UC-22, UC-25). Figure 3.7 shows buying and picking up a surplus bag. The stock is reduced in a single database step, so two buyers can never get the last bag. At pickup, the Orders module checks the code against the branch and asks the Impact ledger to record the saving, exactly once.

Send unsold food to an NGO (UC-33). This flow is started by the Scheduler, not by a user:

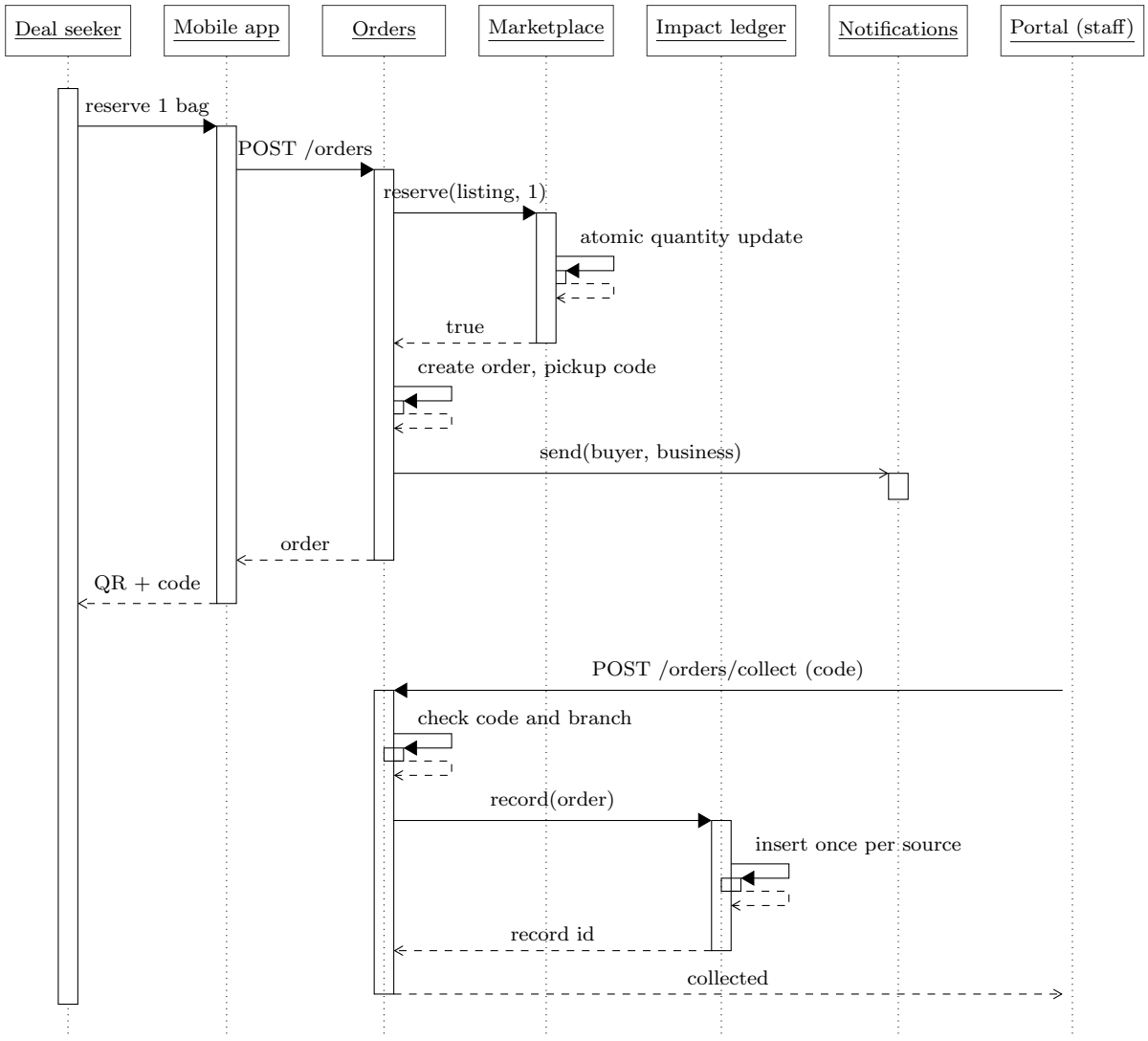


Figure 3.7: UML sequence diagram: reserving a bag and collecting it with the pickup code (SRS UC-22 and UC-25).

1. When a listing is created, the Scheduler sets a job for the end of its pickup window.
2. At that time, the job asks the Marketplace (`IStock.unsoldAtWindowEnd`) what is left. If food is left and the business has auto-donation on, it passes it to Donations.
3. Donations finds verified NGOs nearby whose food types, capacity and opening hours match, putting the business's preferred NGO first.
4. It sends an offer to the first NGO and gives it 30 minutes. If the NGO declines or does not answer, the offer goes to the next NGO.
5. When an NGO accepts, the business is notified. The handover is confirmed with a code and recorded in the Impact ledger.

3.3 Design Rationale

3.3.1 Architectures Considered

We compared four options before choosing the design above.

Architecture	Advantages	Why we did not choose it
Microservices (a separate service for each feature)	Each service can be deployed and scaled on its own	Too much setup and running work for five students in one semester (many deployments, network calls between services, keeping stock and impact consistent across services). Our modular monolith keeps the same separation without that cost.
Supabase only (database rules and Edge Functions, no NestJS backend)	Very fast to start; no server to manage	Our business rules would be split between database security rules and many small functions, which is hard to test and debug. The AI gateway, payments and scheduled donations fit better in one backend we control. We still use Supabase for the database, login, storage and live updates.
Two separate backends (one for the app, one for the portal)	Each app gets an API made just for it	Every business rule would be written twice, but the same rules must apply to both apps.
Chosen: modular monolith + Python ML service	One backend that is simple to run, with clear module boundaries; ML in Python, which has the best libraries	Trade-off: the whole backend is deployed together, so a bug in one module can affect others. We reduce this risk with automated tests and module interfaces that let us split a module out later.

3.3.2 Open Design Question

- Which hosted LLM to use. We will choose it in Sprint 1 by comparing accuracy and cost on our test set of Egyptian receipts (SRS TBD-01).

4. Data Design

4.1 Data Description

4.1.1 Data Stores

Store	Contents	Technology
Main database	All business entities (Figures 4.1 and 4.2), locations, settings, impact ledger	Supabase PostgreSQL with PostGIS; one schema per backend module
Job queue	Scheduled and retried jobs	pg-boss tables in the same Supabase database
Offline store (phone)	The household's pantry, catalogue cache, queued changes	SQLite through Drift
File storage	Organisation documents; listing photos; AI photos (deleted within 24 h)	Supabase Storage (private buckets)
ML artefacts	Trained shelf-life and recommendation models, feature snapshots	Files in the ML service's storage bucket, versioned by date
AI test set and example store	Labelled receipts, photos and voice notes (US-14.02); confirmed user corrections used as worked examples, without personal data	Private repository folder; `AIExample` table
Logs and metrics	Request logs, errors, AI call logs (no personal data)	Platform logging; `AIRequestLog` table

4.1.2 From Information Domain to Data Structures

Each real-world thing in the SRS (a person, a household, a pantry item, a business, a listing, an order, a donation, a saving) becomes one class in the diagrams below and one table in PostgreSQL. Each class belongs to exactly one backend module (Section 3.2.1); other modules can read or change it only by calling that module's interface, never by touching its tables.

Figure 4.1 shows the household side and the impact ledger. Figure 4.2 shows organisations, the marketplace and donations. There, *Business* and *NGO* both inherit from *Organisation*, and a business is made up of its branches, each with its own listings.

4.1.3 Key Data Design Decisions

- **Locations.** Listings, branches, NGOs and drop-off points store their location as a PostGIS point with a location index, so "near me" searches take under 2 s (NFR-03). Other users only ever see a distance and an area name, never someone's exact location.

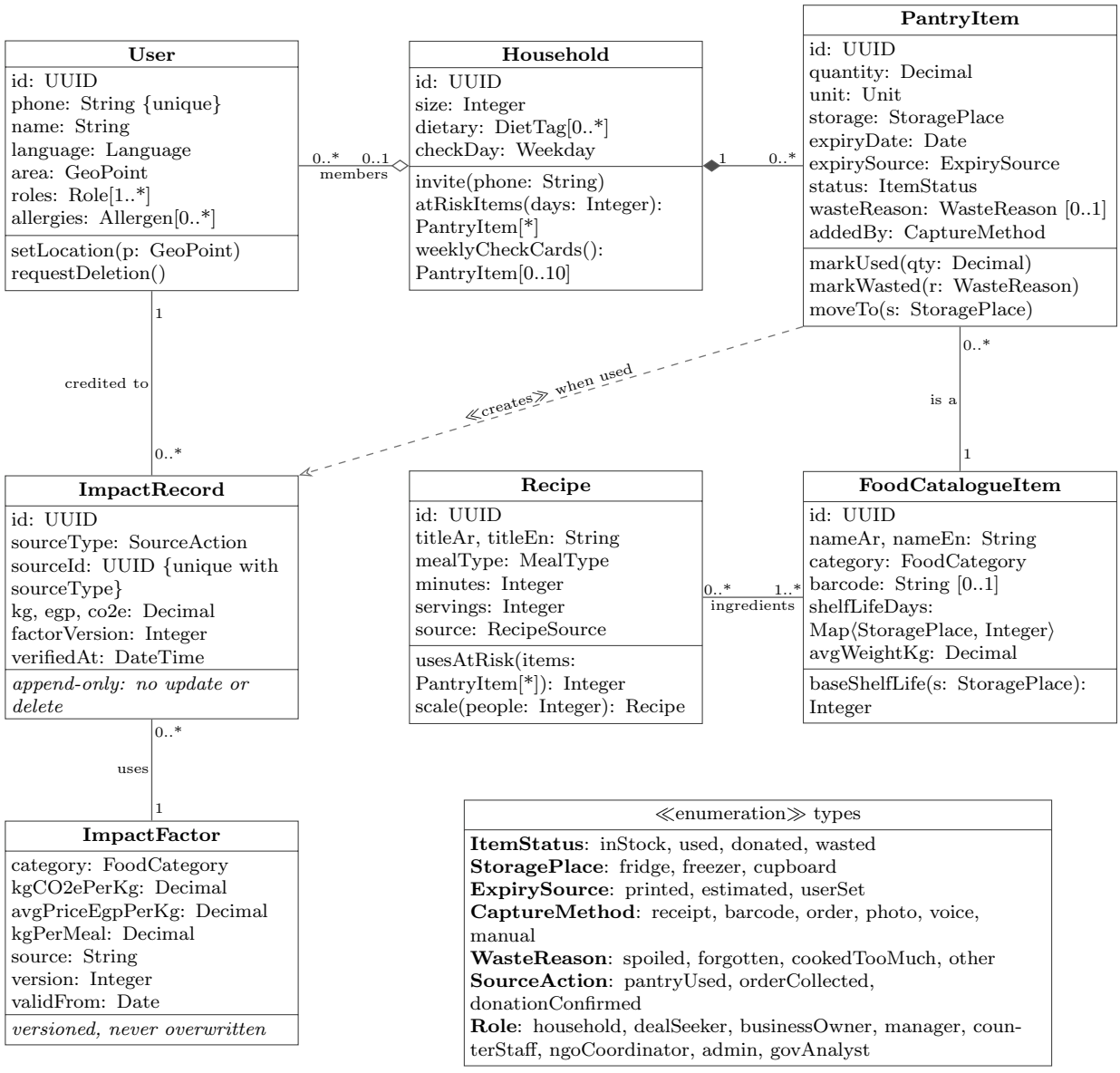


Figure 4.1: UML class diagram 1: users, households, the pantry, recipes and the impact ledger. Filled diamond = “is made of” (composition: the part is deleted with the whole); hollow diamond = “has” (aggregation: the part can exist alone).

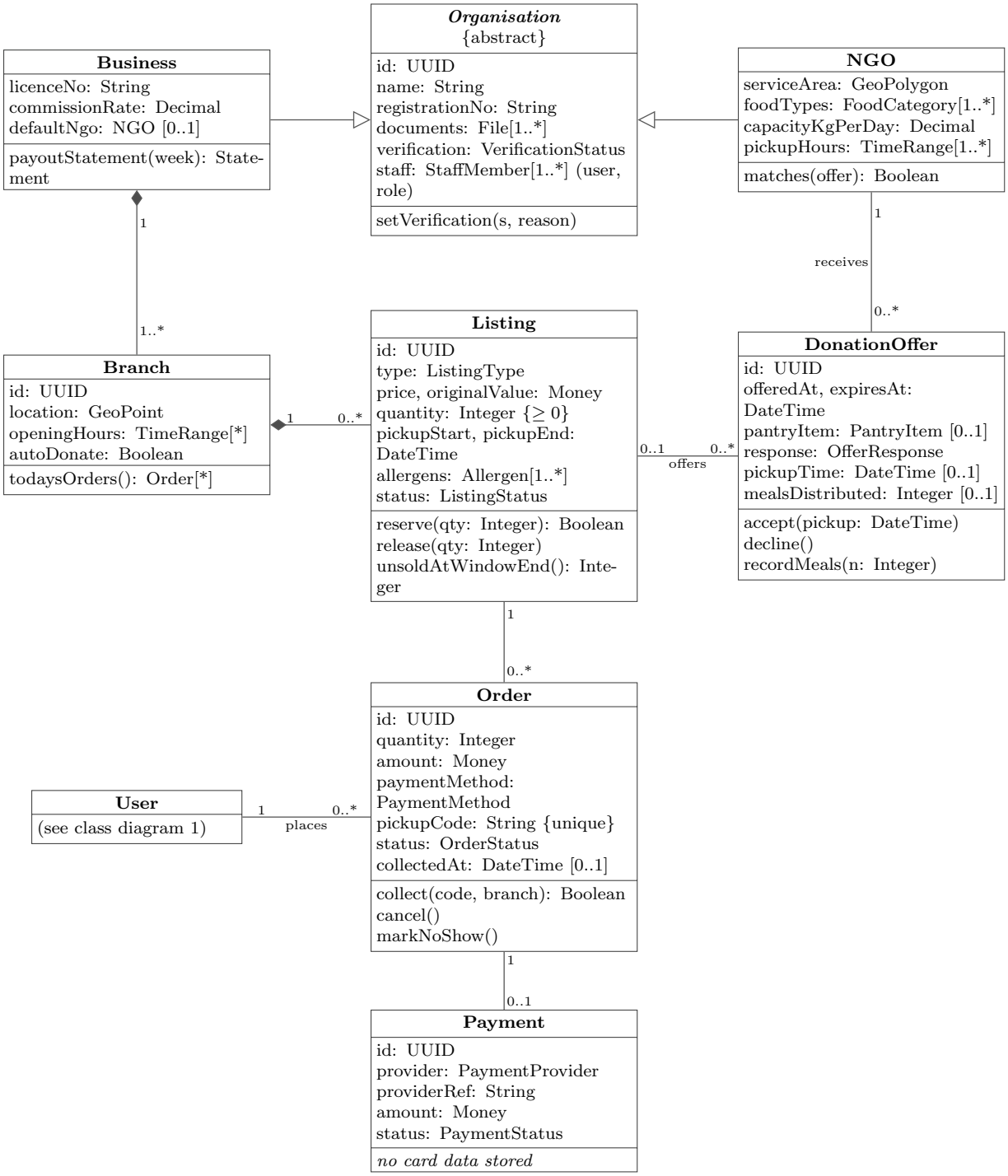


Figure 4.2: UML class diagram 2: organisations, marketplace and donation. A hollow triangle means "is a kind of" (inheritance); *Organisation* is abstract, so it has no objects of its own.

- **Impact ledger.** An `ImpactRecord` is only ever added, never edited or deleted, and there is at most one per action (an order, a donation or a used pantry item), so nothing is counted twice (US-9.03). Each record remembers which version of the conversion factors it used, so old totals do not change when we update the factors. Factors are saved as new versions, never overwritten (US-9.02).
- **Stock.** A reservation reduces `Listing.quantity` in a single database step that only succeeds if enough bags are left. If two people try to buy the last bag at the same moment, only one succeeds (NFR-06).
- **Offline copy.** The mobile app keeps a copy of the household's pantry on the phone (SQLite). Changes made offline are queued with the time they were made and sent when the phone reconnects; if the same field was changed in two places, the latest change wins (NFR-21, R-13).
- **Personal data.** We never store card details (NFR-11). Photos sent to AI are deleted by a Scheduler job once they are 24 hours old (NFR-14). Storage buckets are private, and the apps get short-lived links to view files. AI logs contain no personal data. When a user deletes their account, their personal details are removed within 30 days and only anonymous impact numbers are kept (US-1.08).
- **Jobs.** Scheduled jobs are stored in the same PostgreSQL database (pg-boss), so a job and the data it changes are saved together, and failed jobs are retried (NFR-19).
- **Backups.** The database is backed up, encrypted, every day. During the course a nightly GitHub Actions job exports it with the Supabase CLI, because the free plan has no automatic backups; at launch we use the paid plan's daily backups. At most 24 hours of data can be lost, and the system can be restored within 4 hours (NFR-20).

4.2 Data Dictionary

The main classes in alphabetical order, with their attributes and types and the module that owns them. Every class also has `createdAt` and `updatedAt` timestamps, not repeated below. The fixed value lists (enumerations) are shown in Figure 4.1.

Class	Attributes (name: type)	Description and owner
AIRequestLog	id: UUID; feature: AITask; modelVersion: String; promptVersion: Integer; latencyMs: Integer; costUsd: Decimal; confidence: Decimal; fallbackUsed: Boolean; userCorrected: Boolean; createdAt: DateTime	One row per AI call; holds no personal data (US-14.04, 14.05). *Owner: AI gateway.*
Branch	id: UUID; business: Business; name: String; location: GeoPoint; address: String; openingHours: TimeRange[*]; autoDonate: Boolean	A business location that lists surplus (US-1.06, 6.07). *Owner: Accounts.*

Class	Attributes (name: type)	Description and owner
Business	(inherits Organisation) licenceNo: String; commissionRate: Decimal; defaultNgo: NGO [0..1]	Food business; may list only when verified (BR-01). *Owner: Accounts.*
DonationOffer	id: UUID; ngo: NGO; listing: Listing [0..1]; pantryItem: PantryItem [0..1]; offeredAt, expiresAt: DateTime; response: OfferResponse; pickupTime: DateTime [0..1]; mealsDistributed: Integer [0..1]	An offer of unsold or donated food to one NGO (US-8.02, 8.04, 8.07). *Owner: Donations.*
FoodCatalogue-Item	id: UUID; nameAr, nameEn: String; category: FoodCategory; barcode: String [0..1]; shelfLifeDays: Map(StoragePlace → Integer); avgWeightKg: Decimal	Reference food data managed by administrators (US-13.06). *Owner: Expiry.*
Household	id: UUID; owner: User; members: User[1..*]; size: Integer; dietary: DietTag[*]; checkDay: Weekday	A shared pantry and its members (US-1.05). *Owner: Accounts.*
ImpactFactor	category: FoodCategory; kgCO2ePerKg: Decimal; avgPriceEgpPerKg: Decimal; kgPerMeal: Decimal; source: String; version: Integer; validFrom: Date	Versioned conversion factors; never overwritten (US-9.02). *Owner: Impact ledger.*
ImpactRecord	id: UUID; sourceType: SourceAction; sourceId: UUID; user: User [0..1]; business: Business [0..1]; ngo: NGO [0..1]; category: FoodCategory; kg, egp, co2e: Decimal; meals: Decimal; factorVersion: Integer; verifiedAt: DateTime	One verified saving; unique on (sourceType, sourceId); append-only (US-9.03). *Owner: Impact ledger.*
Listing	id: UUID; branch: Branch; type: ListingType; title: String; price, originalValue: Money; quantity: Integer (≥ 0); pickupStart, pickupEnd: DateTime; allergens: Allergen[1..*]; photoUrl: URL; status: ListingStatus	Surplus offered for pickup (US-6.01, 5.04). *Owner: Marketplace.*
NGO	(inherits Organisation) serviceArea: GeoPolygon; foodTypes: FoodCategory[1..*]; capacityKgPerDay: Decimal; pickupHours: TimeRange[1..*]	Verified charity or food bank (US-1.07, 8.01). *Owner: Accounts, Donations.*
Notification	id: UUID; user: User; type: NotificationType; title, body: String; deepLink: String; sentAt: DateTime; readAt: DateTime [0..1]	In-app and push notification; kept 30 days (US-11.05). *Owner: Notifications.*
Order	id: UUID; buyer: User; listing: Listing; quantity: Integer; amount: Money; paymentMethod: PaymentMethod; pickupCode: String (unique, 4 digits + QR token); idempotencyKey: String (unique); status: OrderStatus; collectedAt: DateTime [0..1]	A reservation of surplus (US-5.04, 7.03). *Owner: Orders.*
Organisation	*abstract*; id: UUID; name: String; registrationNo: String; documents: File[1..*]; verification: VerificationStatus; staff: StaffMember[1..*]	Generalisation of Business and NGO (US-13.01). *Owner: Accounts.*
PantryItem	id: UUID; household: Household; catalogueItem: FoodCatalogueItem; quantity: Decimal; unit: Unit; storage: StoragePlace; expiryDate: Date; expirySource: ExpirySource; status: ItemStatus; wasteReason: WasteReason [0..1]; addedBy: CaptureMethod; updatedAt: DateTime	One food item in a household pantry (E2). *Owner: Pantry.*

Class	Attributes (name: type)	Description and owner
Payment	id: UUID; order: Order; provider: PaymentProvider; providerRef: String; amount: Money; status: PaymentStatus	Online payment record; no card data (NFR-11). *Owner: Orders.*
ReceiptDraft	id: UUID; household: Household; lines: String[*] (masked); items: DraftItem[*] (name, qty, unit, category, confidence); createdAt: DateTime	AI-extracted items awaiting user review; deleted after 24 h (US-2.02). *Owner: Pantry.*
Recipe	id: UUID; titleAr, titleEn: String; ingredients: (FoodCatalogueItem, quantity)[1..*]; steps: String[*]; mealType: MealType; minutes: Integer; servings: Integer; source: RecipeSource (curated or LLM)	Curated Egyptian recipe or checked LLM recipe (E4). *Owner: Recipes.*
Setting	key: String; value: JSON; version: Integer; changedBy: User; changedAt: DateTime	Configurable rule such as discount limits or time limits (US-13.04). *Owner: Administration.*
User	id: UUID; authUid: String (unique, Supabase Auth); phone: String (unique); name: String; language: Language; area: GeoPoint; roles: Role[1..*]; allergies: Allergen[*]; notificationPrefs: JSON; deletedAt: DateTime [0..1]	Any registered person (E1). *Owner: Accounts.*

5. Human Interface Design

5.1 Overview of User Interface

Mobile app (households and deal seekers). The app opens on *Home*, which shows the items closest to expiry, today's deals nearby and the user's impact this week. A bottom bar gives five tabs:

- **Home**: at-risk items with one-tap *recipe* and *donate* actions; recipe suggestions (*Use it up*).
- **Pantry**: items grouped by fridge, freezer and cupboard and coloured by urgency. A large *Add food* button opens the capture methods: scan receipt, scan barcode, photo, voice and manual. After any AI capture the user sees a review screen and confirms with one tap. Once a week the pantry offers the 30-second swipe check.
- **Deals**: a map and a list of nearby surplus; a listing page with price, pickup window, allergens and *Reserve*; after reserving, a QR code and 4-digit code for pickup.
- **Donate**: choose sealed pantry items to donate, see partner drop-off points on a map, or buy a bag for an NGO.
- **Impact**: kilograms, pounds, CO_2e and meals saved; the monthly waste rate against the Egyptian average; *How we calculate*.

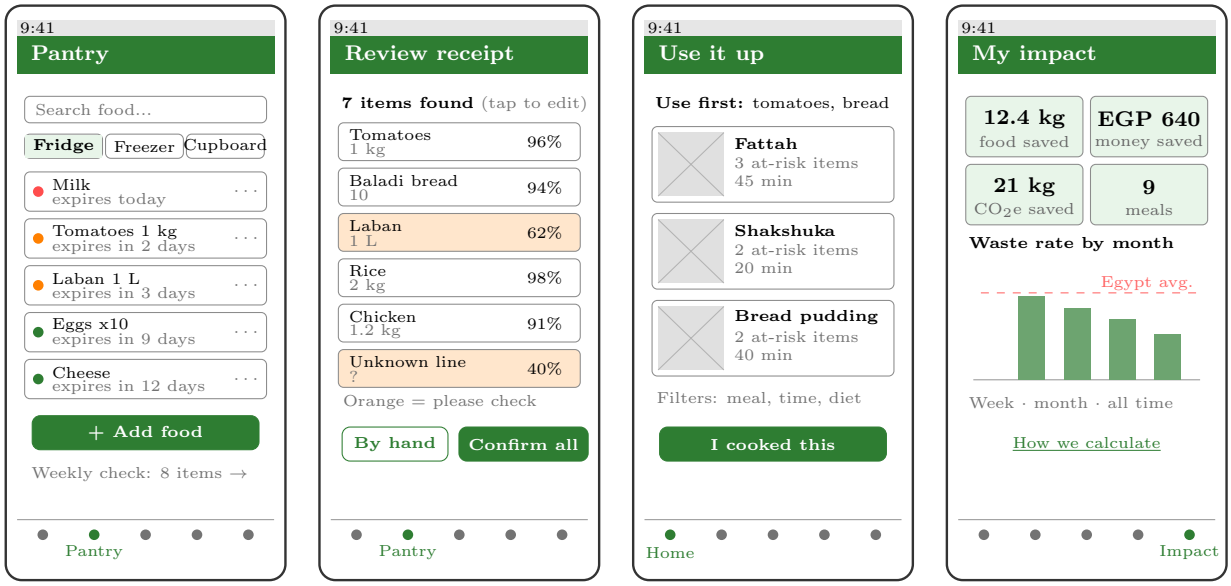
Web portal (businesses, NGOs, administrators, analysts). After sign-in the portal shows only the areas of the user's role in a left-hand menu. Businesses see *Today's orders* (with *Scan pickup code*), *Listings*, *Donations*, *Impact* and *Settings*. NGOs see *Offers*, *Pickups* and *Impact*. Administrators see *Verification*, *Moderation*, *Disputes*, *Settings* and *Analytics*. Analysts see *Exports*.

Feedback to the user. Every action gets visible feedback: a short message at the bottom of the screen for success ("7 items added"), inline errors next to the field, a spinner with text for AI work longer than 1 s ("Reading your receipt…"), confidence percentages and orange highlighting on uncertain AI results, an offline banner, and an *Undo* for 10 s after automatic pantry changes. Push notifications open the related screen directly.

Language and accessibility. Arabic with a right-to-left (mirrored) layout is the default and English is available; the wireframes below are shown in English for readability. Every control has a text label and a screen-reader label, buttons are large enough to tap easily (at least 48 dp), and colour is never the only signal (SRS NFR-16 to NFR-18).

5.2 Screen Images

Figures 5.1 and 5.2 are low-fidelity wireframes of the main screens for the stories we implement during the course. They show layout and content, not final visual design. We will replace them with screenshots of the real app as each sprint is delivered.



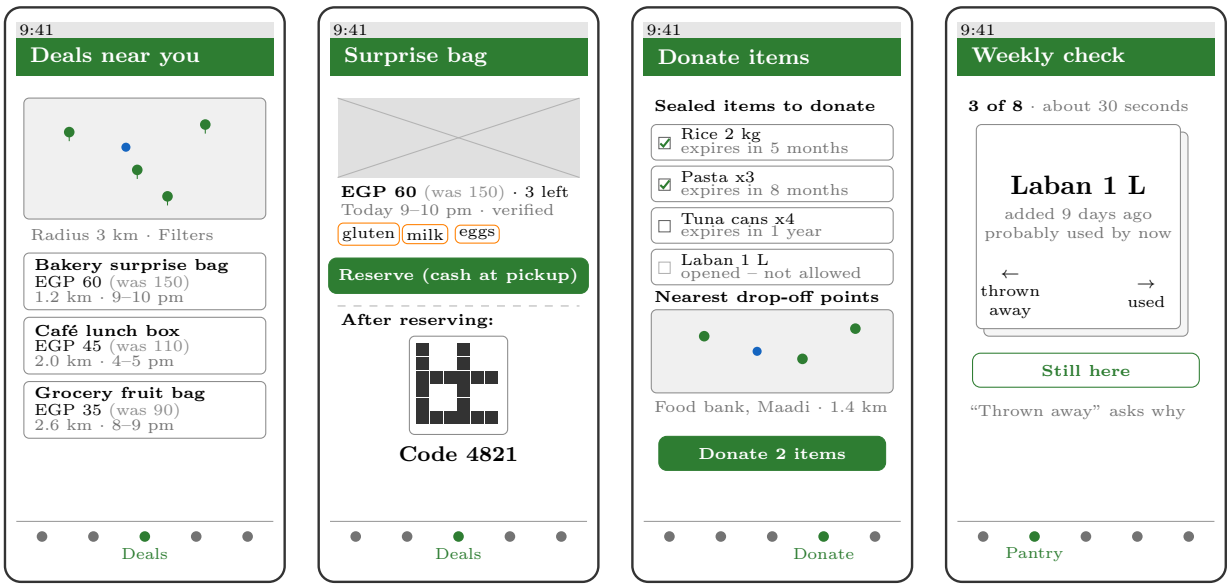
(a) Pantry (US-2.11) (b) Receipt review (US-2.02) (c) Recipe ideas (US-4.01) (d) Impact (US-9.01)

Figure 5.1: Wireframes of the household screens (mobile app).

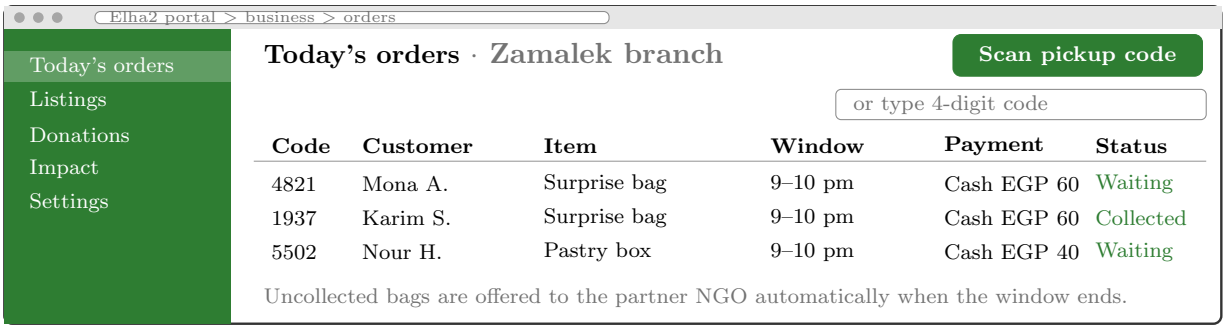
5.3 Screen Objects and Actions

Screen	Object	Action and system response	Stories
(a) Pantry	Storage tabs	Tap to filter by fridge, freezer or cupboard	US-2.11
	Item row with urgency dot	Red = expires today, orange = within 3 days, green = later. Tap opens the item with recipe, donate, move and “thrown away” actions	US-2.11, 2.10, 3.06
	Add food button	Opens the capture sheet: receipt, barcode, photo, voice, manual	US-2.01–2.07
	Weekly check link	Opens the swipe check (h)	US-2.09
(b) Review receipt	Item row with confidence	Tap to edit name, quantity, unit, storage or expiry; swipe to delete; orange = low confidence	US-2.02
	Confirm all	Saves every item to the pantry and shows “7 items added”	US-2.02
	By hand	Opens manual entry with autocomplete; used also when the AI fails	US-2.07
(c) Use it up	Recipe card	Tap to open steps scaled to the household	US-4.01

Screen	Object	Action and system response	Stories
	I cooked this	Deducts ingredients, adds impact, offers *Undo* for 10 s	US-4.05, 2.08
(d) My impact	Four totals	Tap a total to see it by Prevent, Sell and Donate; switch week, month, all time	US-9.01
	Waste-rate chart	Shows monthly rate against the Egyptian average line	US-9.04
	How we calculate	Opens the factors, sources and version dates	US-9.02
(e) Deals	Map pins and list cards	Tap a pin or card to open the listing; radius and filters at the top	US-5.01, 5.03
(f) Surprise bag	*Reserve*	Reserves the bag (cash at pickup) and shows the QR code and 4-digit code; "sold out" if the last bag was just taken	US-5.04, 7.02, 7.03
	Directions	Opens Google Maps to the store	US-5.07
(g) Donate items	Sealed-item checklist	Only sealed, unexpired pantry items can be ticked	US-8.06
	Drop-off map	Shows the nearest partner drop-off points and their opening hours	US-8.06
	Donate	Records the donation; the NGO confirms receipt, then the items leave the pantry and impact is added	US-8.06, 2.08
(h) Weekly check	Card stack	Swipe right = used, left = thrown away (asks for a reason), button = still here	US-2.09, 2.10
(i) Portal: today's orders	*Scan pickup code*	Opens the camera; a valid code marks the order collected and turns its status green	US-6.04
	Code field	Type the 4-digit code instead of scanning	US-6.04



(e) Deals (US-5.01) (f) Reserve and pickup (US-5.04) (g) Donate items (US-8.06) (h) Weekly check (US-2.09)



(i) Business portal: today's pickups (US-6.04), counter-staff view

Figure 5.2: Wireframes of the marketplace and donation screens (mobile app) and the business pickup screen (web portal).

6. Requirements Matrix

This chapter links the design to the SRS in both directions. Every feature (E1 to E14), user story and non-functional requirement is handled by at least one component, and every component exists because of at least one requirement. The tests for each requirement are listed in the SRS verification matrix (SRS Appendix D). When a requirement changes, these tables show which components to update; when a component changes, they show which requirements to test again.

6.1 Components to Requirements

Component	Functional requirements realised	Non-functional requirements
Mobile app	E1, E2, E3, E4, E5, E7, E8, E9, E10, E11	NFR-01, NFR-04, NFR-16 to NFR-18, NFR-21, NFR-27, NFR-28, NFR-30, NFR-31
Web portal	E6, E8, E9, E13	NFR-17, NFR-27, NFR-31
API layer	E1–E14 (authentication, validation, routing)	NFR-02, NFR-05, NFR-09, NFR-10, NFR-12, NFR-23 to NFR-26
Accounts	E1, E10 (referrals)	NFR-10, NFR-13
Pantry	E2, E3 (digests), E4 (ingredient deduction)	NFR-04, NFR-21
Expiry	E3	–
Recipes	E4, E10 (Ramadan recipes, portion planner)	NFR-07
Marketplace	E5, E6, E12 (allergens, listing screening)	NFR-03, NFR-06
Orders	E7	NFR-06, NFR-11
Donations	E8 (including household donations)	NFR-19
Administration	E12, E13	NFR-08
AI gateway	E14; AI requirements in E2, E4, E6, E9	NFR-07, NFR-14, NFR-22, NFR-29, NFR-30
Impact ledger	E9, E10 (badges, streaks, challenges)	NFR-15
Notifications	E11, E10 (Ramadan prompts)	–
Scheduler	E3, E8 (timed jobs)	NFR-19
Adapters	E2 (barcodes, email), E5 (maps), E7 (payments)	NFR-11
ML service	E3 (shelf life), E5 (US-5.06)	NFR-29
Supabase (database, Auth, Storage, Realtime)	All epics (persistence, login, files, live updates)	NFR-03, NFR-06, NFR-09, NFR-20

6.2 Functional Requirements to Components and Data

The table lists every functional requirement in the SRS by its identifier, the components that satisfy it (Chapter 3) and the main classes it reads or writes (Chapter 4).

Req.	Title	Components	Main classes
US-1.01	Sign up with a phone number	Mobile app; Accounts	User, Household
US-1.02	Switch the app language	Mobile app; Accounts	User, Household
US-1.03	Complete a short onboarding	Mobile app; Accounts	User, Household
US-1.04	Set a location	Mobile app; Accounts; Adapters (Maps)	User
US-1.05	Share a household pantry	Mobile app; Accounts	Household, User
US-1.06	Register a business	Web portal; Accounts	Business, Branch
US-1.07	Register an NGO	Web portal; Accounts	NGO
US-1.08	Delete an account	Mobile app; Accounts; Scheduler	User
US-2.01	Scan a receipt [**AI**]	Mobile app (OCR, masking); Pantry; AI gateway; Expiry	ReceiptDraft, PantryItem
US-2.02	Review AI-extracted items [**AI**]	Mobile app; Pantry	ReceiptDraft, PantryItem
US-2.03	Scan a barcode [**AI**]	Mobile app (ML Kit); Pantry; Adapters (Open Food Facts)	FoodCatalogueItem, PantryItem
US-2.04	Import online grocery orders [**AI**]	Adapters (inbound email); Pantry; AI gateway	ReceiptDraft, PantryItem
US-2.05	Photograph fridge or groceries [**AI**]	Mobile app; Pantry; AI gateway	ReceiptDraft, PantryItem
US-2.06	Log items by voice [**AI**]	Mobile app; Pantry; AI gateway (speech, LLM)	ReceiptDraft, PantryItem
US-2.07	Add an item manually	Mobile app; Pantry	PantryItem
US-2.08	Update pantry automatically	Pantry; Recipes; Donations	PantryItem
US-2.09	Complete the weekly check [**AI**]	Mobile app; Pantry; Scheduler	PantryItem
US-2.10	Record thrown-away food	Mobile app; Pantry; Impact ledger	PantryItem
US-2.11	View the pantry	Mobile app; Pantry	PantryItem
US-3.01	Estimate an expiry date [**AI**]	Expiry; ML service	PantryItem, FoodCatalogueItem
US-3.02	Read a printed date [**AI**]	Mobile app (OCR); Expiry; AI gateway	PantryItem
US-3.03	Receive a daily expiry digest	Pantry; Scheduler; Notifications	PantryItem, Notification
US-3.04	Explain date labels	Mobile app; Expiry	PantryItem
US-3.05	Show at-risk items on the home screen	Mobile app (widget)	PantryItem
US-3.06	Act on an expiring item	Mobile app; Notifications	PantryItem
US-4.01	Suggest recipes for at-risk food [**AI**]	Mobile app; Recipes; Pantry; AI gateway	Recipe, PantryItem
US-4.02	Ask the cooking assistant [**AI**]	Mobile app; Recipes; AI gateway	Recipe, PantryItem
US-4.03	Filter recipes	Mobile app; Recipes	Recipe
US-4.04	Scale a recipe [**AI**]	Recipes; AI gateway	Recipe
US-4.05	Mark a meal as cooked	Mobile app; Recipes; Pantry; Impact ledger	PantryItem, ImpactRecord
US-4.06	Save and share recipes	Mobile app; Recipes	Recipe
US-4.07	Get leftover ideas [**AI**]	Recipes; AI gateway; Scheduler	Recipe
US-5.01	Browse nearby surplus	Mobile app; Marketplace; Supabase (PostGIS)	Listing, Branch
US-5.02	Filter listings	Mobile app; Marketplace	Listing

Req.	Title	Components	Main classes
US-5.03	View a listing	Mobile app; Marketplace	Listing
US-5.04	Reserve a bag	Mobile app; Orders; Marketplace	Order, Listing
US-5.05	Follow a store	Marketplace; Notifications	Listing, Notification
US-5.06	See a personal feed **[AI]**	Marketplace; ML service	Listing, Order
US-5.07	Get directions	Mobile app; Adapters (Maps)	Branch
US-5.08	Rate a store	Mobile app; Orders	Order
US-5.09	Report a problem with an order	Mobile app; Orders; Administration	Order
US-6.01	Create a listing	Web portal; Marketplace	Listing
US-6.02	Fill a listing from a photo **[AI]**	Web portal; Marketplace; AI gateway	Listing
US-6.03	Manage staff roles	Web portal; Accounts	Organisation, Branch
US-6.04	Confirm a pickup	Web portal; Orders; Impact ledger	Order, ImpactRecord
US-6.05	Edit or cancel a listing	Web portal; Marketplace; Orders; Notifications	Listing, Order
US-6.06	Import stock in bulk	Web portal; Marketplace	Listing
US-6.07	Turn on auto-donation	Web portal; Accounts; Donations	Branch
US-7.01	Pay in the app	Mobile app; Orders; Adapters (Paymob)	Order, Payment
US-7.02	Pay cash at pickup	Orders	Order
US-7.03	Show a pickup code	Mobile app; Orders; Impact ledger	Order, ImpactRecord
US-7.04	Cancel a reservation	Mobile app; Orders; Marketplace; Scheduler	Order, Listing
US-7.05	Refund a business cancellation	Orders; Adapters (payments); Notifications	Order, Payment
US-7.06	View order history	Mobile app; Orders	Order
US-7.07	Receive payouts	Web portal; Orders; Adapters (payments); Scheduler	Order, Payment
US-8.01	Set donation preferences	Web portal; Accounts; Donations	NGO
US-8.02	Route unsold food to an NGO	Donations; Scheduler; Notifications; Marketplace	DonationOffer, Listing
US-8.03	Plan a collection route **[AI]**	Web portal; Donations; Adapters (Maps)	DonationOffer
US-8.04	Accept a donation	Web portal; Donations	DonationOffer, NGO
US-8.05	Donate a bag	Mobile app; Orders; Donations	Order, DonationOffer
US-8.06	Donate pantry items	Mobile app; Donations	DonationOffer, PantryItem
US-8.07	Record meals distributed	Web portal; Donations; Impact ledger	DonationOffer, ImpactRecord
US-8.08	Issue a donation receipt	Web portal; Donations; Impact ledger	DonationOffer
US-9.01	View personal impact	Mobile app; Impact ledger	ImpactRecord
US-9.02	Explain the calculation	Mobile app; Impact ledger	ImpactFactor
US-9.03	Count only verified actions	Impact ledger	ImpactRecord, ImpactFactor
US-9.04	View the waste rate	Mobile app; Impact ledger; Pantry	ImpactRecord, PantryItem
US-9.05	Get a monthly insight **[AI]**	Impact ledger; AI gateway; Scheduler	ImpactRecord, PantryItem
US-9.06	Set a waste goal	Mobile app; Impact ledger; Notifications	ImpactRecord
US-9.07	View the business dashboard	Web portal; Impact ledger	ImpactRecord, Listing
US-9.08	Export a CSR report	Web portal; Impact ledger	ImpactRecord
US-9.09	View the NGO dashboard	Web portal; Impact ledger	ImpactRecord, DonationOffer
US-9.10	Export anonymised impact data	Web portal; Impact ledger	ImpactRecord
US-9.11	Share an impact card	Mobile app; Impact ledger	ImpactRecord
US-10.01	Earn badges and streaks	Mobile app; Impact ledger	ImpactRecord
US-10.02	Join a neighbourhood challenge	Mobile app; Impact ledger	ImpactRecord

Req.	Title	Components	Main classes
US-10.03	Use Ramadan mode [**AI**]	Mobile app; Recipes; Notifications; Scheduler	Recipe
US-10.04	Plan portions for guests [**AI**]	Mobile app; Recipes; AI gateway	Recipe
US-10.05	Read storage tips	Mobile app; Notifications; Administration	Notification
US-10.06	Invite friends	Mobile app; Accounts	User
US-11.01	Receive push notifications	Mobile app; Notifications; Firebase (FCM)	Notification
US-11.02	Set notification preferences	Mobile app; Notifications	User, Notification
US-11.03	Get deal alerts	Notifications; Marketplace	Notification
US-11.04	Get a pickup reminder	Notifications; Scheduler; Orders	Notification, Order
US-11.05	View past notifications	Mobile app; Notifications	Notification
US-12.01	Screen content automatically	Marketplace; Administration	Listing
US-12.02	Report a user or content	Mobile app; Administration	Setting
US-12.03	Show a verified badge	Accounts; Mobile app	Organisation
US-12.04	Declare allergens	Mobile app; Marketplace	User, Listing
US-13.01	Verify organisations	Web portal; Administration; Accounts	Organisation
US-13.02	Moderate flagged content	Web portal; Administration	Listing
US-13.03	Resolve disputes	Web portal; Administration; Adapters (payments)	Order, Payment
US-13.04	Configure settings	Web portal; Administration	Setting
US-13.05	View platform analytics	Web portal; Administration	Order, ImpactRecord
US-13.06	Manage catalogue and recipes	Web portal; Administration; Expiry; Recipes	FoodCatalogueItem, Recipe
US-14.01	Route all AI calls through a gateway	AI gateway	AIRequestLog
US-14.02	Measure AI accuracy	AI gateway; CI pipeline	AIRequestLog
US-14.03	Mask personal data	Mobile app (masking)	ReceiptDraft
US-14.04	Log AI corrections	AI gateway	AIRequestLog
US-14.05	Cap AI cost	AI gateway	AIRequestLog

7. Appendices

Appendix A: Declaration of AI Use

The design in this document was decided by our team, building on our earlier project work and the SRS. We used Claude (Anthropic) to help structure and draft this document, its diagrams and its LaTeX layout. All content was reviewed and edited by us, and we take full responsibility for the final document.